Building a Scalable Sitemap Prerendering System

Using Cloudflare Workers, Puppeteer & KV Storage

Introduction

Modern web applications are predominantly built as Single Page Applications (SPAs) frameworks like React, Vue, and Angular deliver rich, interactive experiences by rendering HTML entirely in the browser using JavaScript. While this creates excellent user experiences, it creates a critical SEO problem: search engine bots and social media crawlers cannot reliably execute JavaScript, meaning they often see an empty page.

This guide walks through building a production-grade prerendering system that solves this problem completely without relying on any paid third-party service. Using Cloudflare Workers at the edge, Puppeteer, and Cloudflare Workers KV for persistent caching, we will build a system that:

- Intercepts bot traffic at the CDN edge before it reaches your origin server.
- Serves fully rendered HTML snapshots from a fast KV cache.
- Stays entirely within Cloudflare's free tier for most use cases.
- Scales to hundreds or thousands of pages using a simple batch crawl strategy.

Prerequisites

- Cloudflare account (free tier is sufficient to start).
- A domain added to Cloudflare with DNS proxied.
- Node.js v18 or higher installed on your local machine.
- Wrangler CLI: npm install -g wrangler.
- Basic familiarity with JavaScript and command-line tools.
- Your website must have a sitemap.xml available at a public URL.

Important: Cloudflare Browser Rendering requires the @cloudflare/puppeteer npm package and must be deployed via Wrangler CLI — it cannot be used from the Cloudflare dashboard editor alone.

Let’s Us Drive into to the process…

Implementation

Step 1: Create a Cloudflare Worker

What is a Worker?

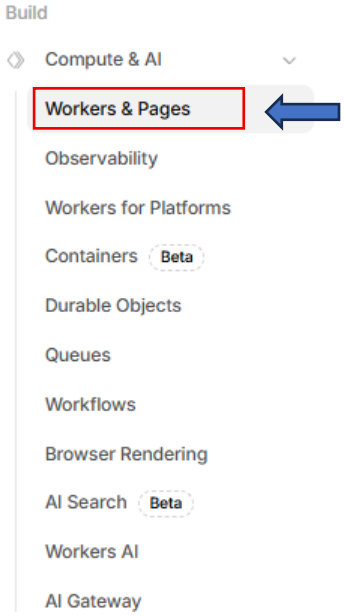
A Cloudflare Worker is a serverless JavaScript function that runs at Cloudflare's edge locations.

It allows us to:

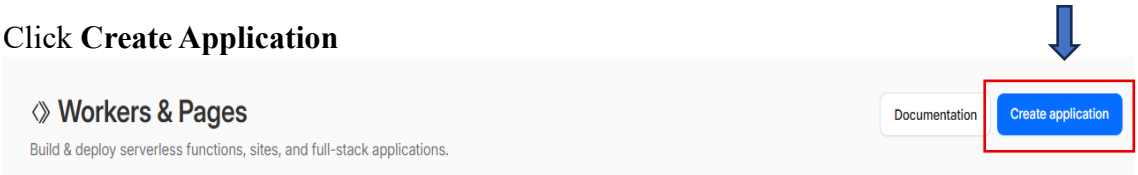
- Intercept HTTP requests
- Read/write KV data and return custom responses

1.1 Navigate to Workers

1. Login to **Cloudflare** Dashboard
2. In Dashboard, Go to **Build → Compute & AI → Workers & Pages**

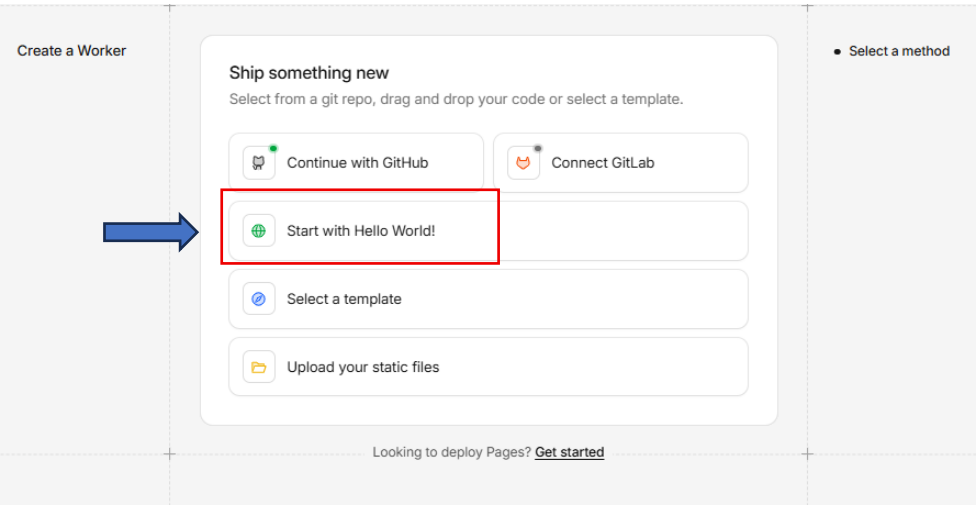


3. Click **Create Application**



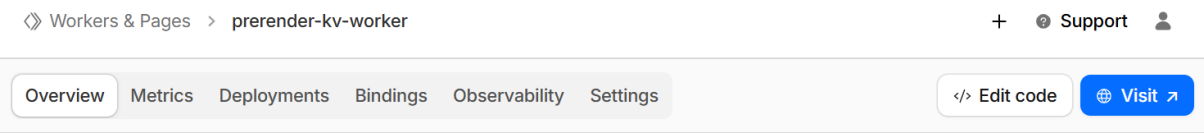
1.2 Create a New Worker

Choose ***"Start* with Hello World"** This creates a basic Worker with a default fetch () handler.



Give your worker a meaningful name, for example: **prerender-kv-worker**

Once created, Cloudflare opens the **Worker Editor**.



STEP 2: Create a Workers KV Namespace

What is a KV Namespace?

A KV namespace stores data as:

key → value

In our case:

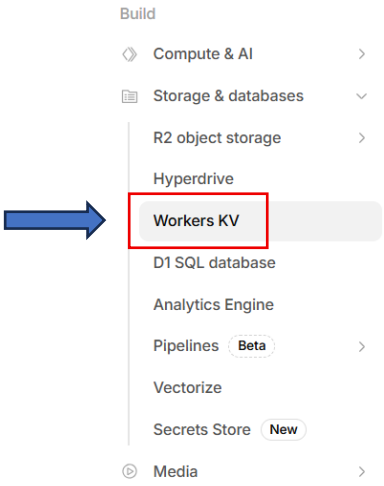
"/about" → "<html>About Page</html>"

Each key represents a URL path, and the value is the pre-rendered HTML.

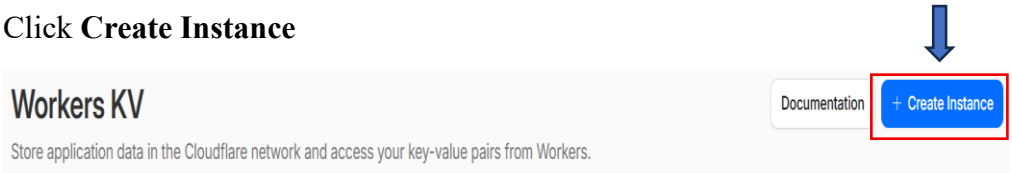
Workers KV is a globally distributed key-value storage optimized for fast reads.

2.1 Create KV Namespace

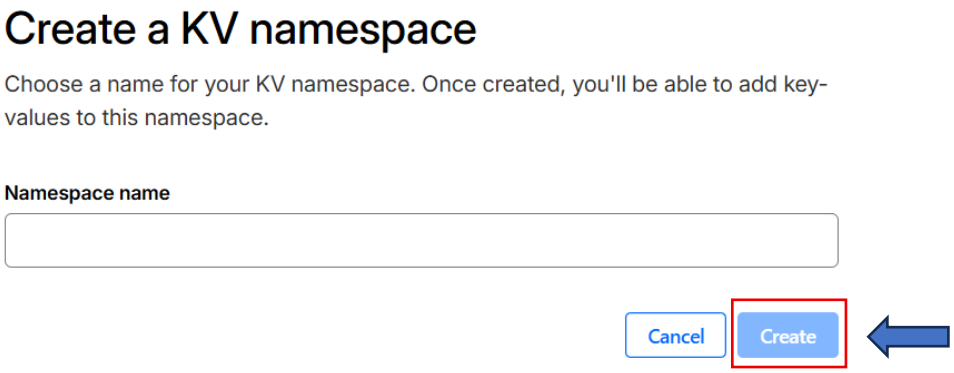
1. Go to **Build → Storage & Databases → Workers KV**



2. Click **Create Instance**



3. Give any Namespace name for e.g. " **PRERENDER_KV_HTML**"



4. Click **Create**, After creation, copy the **Namespace ID** (used internally by Cloudflare).

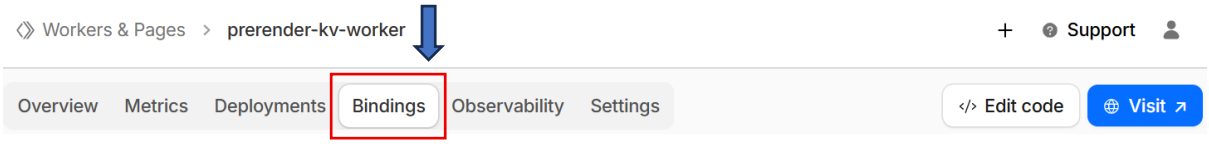
STEP 3: Bind KV Namespace to the Worker

Binding allows the Worker code to access the KV namespace.

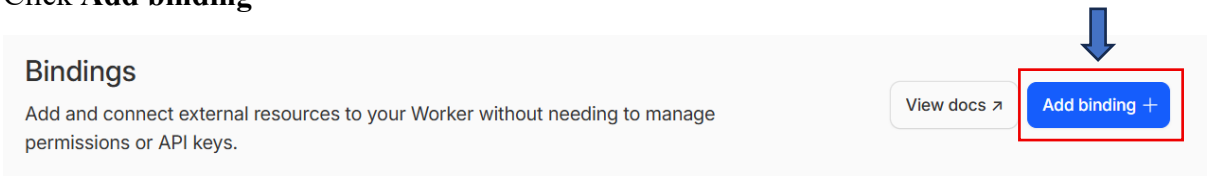
3.1 Add KV Binding

1. Open your Worker

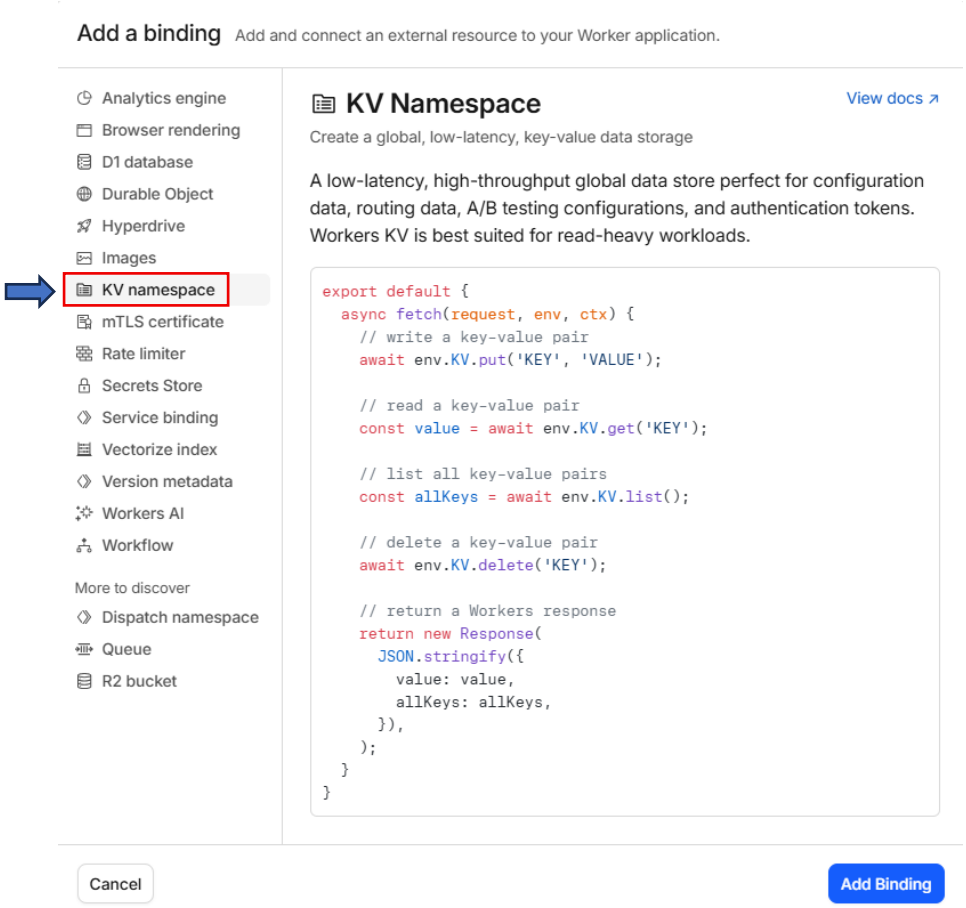
2. Go to **Build → Workers & Pages → Settings → Bindings**



3. Click **Add binding**



4. Choose **KV Namespace**



Fill the fields:

Field	Value
Variable name	Your Variable name
KV namespace	Your KV Namespace

5. After Filling, Click **Add Binding**

Why Variable Name Matters !

The variable name is how you access KV inside JavaScript code:

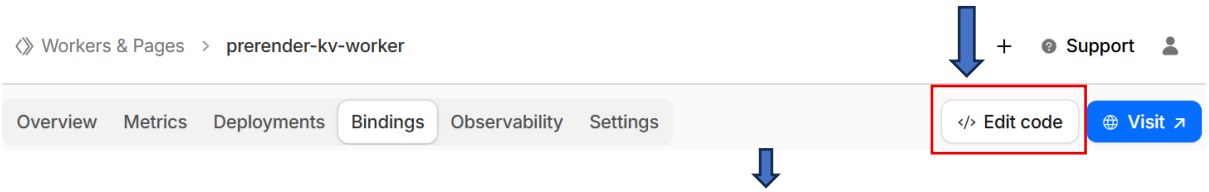
env.PRERENDER_HTML_KV.get("/about")

Namespace name and variable name do not need to match.

STEP 4: Serve HTML Pages from KV Using Worker

4.1 Final Worker Code

Go to **Build → Workers & Pages → Settings → Edit code**



Replace the Default Worker code with the following: Click

How the Code Works

The system takes the URL path (like /about or /contact) from the request and uses it as a key to check in KV storage. Since the homepage / cannot be stored directly, it is converted to /index. If the HTML is found in KV, it is returned instantly. If not, a 404 response is sent.

KV Key Naming Rules

KV keys must match the URL paths exactly. The homepage / is stored as /index, and all other pages like /about or /pricing are stored the same as their paths. This makes fetching data simple and fast.

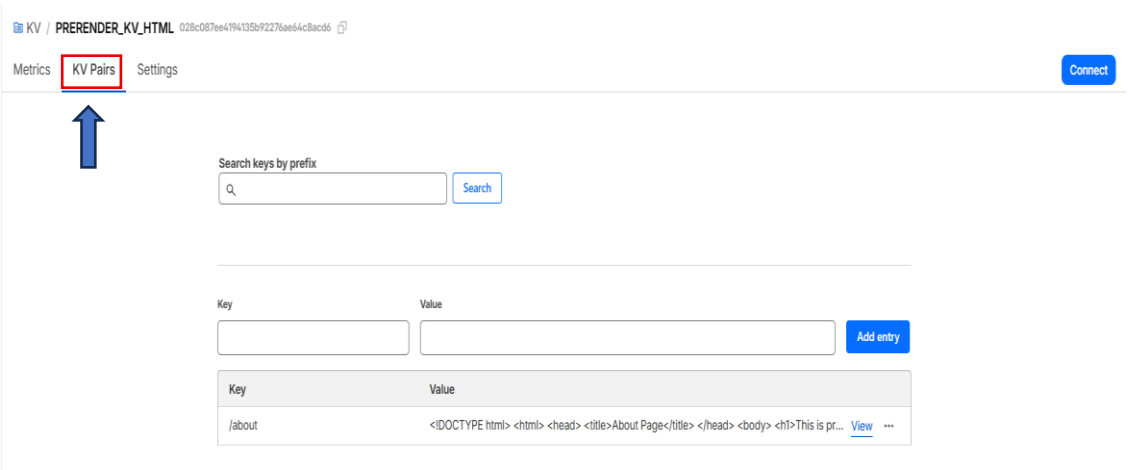
4.2 Testing the Setup

Go to **Build → Storage & Databases → Workers KV**

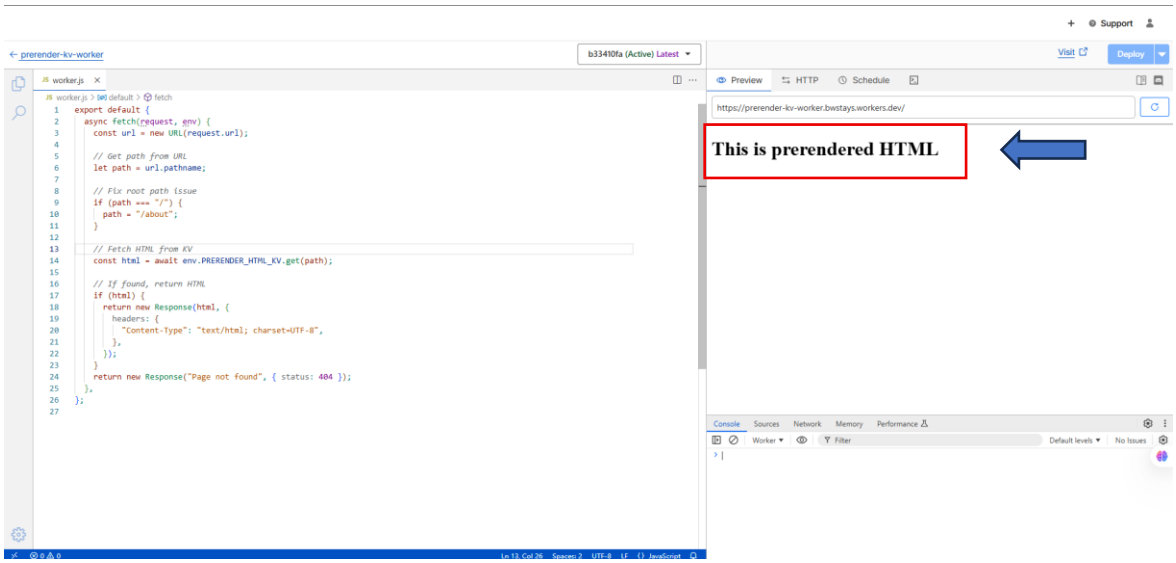
Add test entry in KV

Key: /test

Value: <h1>KV Test Page</h1>



Go to **Workers & Pages → Edit Code → Preview**, you will find the Output:



If the page loads, KV + Worker is working correctly.

Step 5: Manually Store One HTML Page in KV and Serve It

Before doing automation (90 pages, sitemap, bots), we must prove the system works for ONE page.

5.1 Choose ONE Page to Prerender

Pick any page. Start with homepage.
e.g. https://www.bwstays.com

5.2 Copy the Full HTML Source

1. Open e.g. https://www.bwstays.com
2. Right-click → View Page Source (NOT "Inspect")
3. You will see the entire source code:

   Select **ALL** (Ctrl + A), Copy EVERYTHING

This is your prerendered HTML.

5.3 Store HTML in Workers KV

Go to **Storage and Database → workers KV → your KV namespace → KV pairs**

Now we store this HTML in KV.

for e.g.
key as: "/about "
Value: (Paste the full HTML you copied)
click save.

5.5 Test the Worker

Go to **Compute & AI → Workers & Pages → Your workers namespace → Visit**
then Your website homepage loads, there You can observe the prerendered page.

Step 6: Cloudflare Browser Rendering

6.1 Enabling the Browser rendering

Go to your **Cloudflare Dashboard → Account Home**
In the left sidebar, click **Build → Compute → Browser Rendering**
Make sure it's **enabled** for your account

- Cloudflare/puppeteer package needs to be explicitly declared in your worker's configuration. You **cannot** use npm imports directly in the Cloudflare dashboard editor **Deploy via Wrangler CLI (Required for Puppeteer)**

6.2 Install Node.js & Wrangler

Open Your Terminal/ Command prompt and run:

bash:

```
npm install -g wrangler
```

To verify it's installed:

bash:

```
wrangler -version
```

```
C:\WINDOWS\system32\cmd.

Your environment has been set up for using Node.js 24.13.0 (x64) and npm.

C:\Users\ADMIN>wrangler --version

 wrangler 4.62.0 (update available 4.78.0)


C:\Users\ADMIN>npm install -g wrangler@latest

changed 34 packages in 1m

6 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New minor version of npm available! 11.6.2 -> 11.12.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.12.1
npm notice To update run: npm install -g npm@11.12.1
npm notice

C:\Users\ADMIN>npm install -g npm@11.12.1

added 1 package in 10s

15 packages are looking for funding
  run `npm fund` for details

C:\Users\ADMIN>wrangler --version

 wrangler 4.78.0
```

6.3 Login to Cloudflare

bash:

```
wrangler login
```

```
C:\Users\ADMIN>wrangler login

 wrangler 4.78.0

Attempting to login via OAuth...
Opening a link in your default browser: https://dash.cloudflare.com/oauth2/auth?response_type=code&client_id=54d11594-84
e4-41aa-b438-e81b8fa78ee7&redirect_uri=http%3A%2F%2Flocalhost%3A8976%2Foauth%2Fcallback&scope=account%3Aread%20user%3Are
ad%20workers%3Awrite%20workers_kv%3Awrite%20workers_routes%3Awrite%20workers_scripts%3Awrite%20workers_tail%3Aread%20d1%
3Awrite%20pages%3Awrite%20zone%3Aread%20ssl_certs%3Awrite%20ai%3Awrite%20ai-search%3Awrite%20ai-search%3Arun%20queues%3A
write%20pipelines%3Awrite%20secrets_store%3Awrite%20containers%3Awrite%20cloudchamber%3Awrite%20connectivity%3Aadmin%20o
ffline_access&state=HNXZYmAK9Ro5YaEZ0zEOenQI_uGvJ09R&code_challenge=eViukKJ5Rm9nSPKrjMUTzWRz3kW-LT8c1f8K-uuU1u0&code_cha
llenge_method=S256
Successfully logged in.
```

Step 7: Create the Sitemap Crawler Worker Project

7.1 Open Node.js Command Prompt & Create the Project Folder

- Search for "Node.js command prompt" and open.

bash

```
mkdir prerender-sitemap-crawler

cd prerender-sitemap-crawler
```

```
C:\Users\ADMIN>mkdir prerender-sitemap-crawler

C:\Users\ADMIN>cd prerender-sitemap-crawler

C:\Users\ADMIN\prerender-sitemap-crawler>npm init -y
Wrote to C:\Users\ADMIN\prerender-sitemap-crawler\package.json:

{
  "name": "prerender-sitemap-crawler",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

7.2 Initialize the Project and Install Dependencies

Bash:

```
npm init -y

npm install @cloudflare/puppeteer
```

```
C:\Users\ADMIN\prerender-sitemap-crawler>npm install @cloudflare/puppeteer

added 81 packages, and audited 82 packages in 17s

6 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

This will be created:

prerender-sitemap-crawler/

--package.json

--package-lock.json

--node_modules/

Step 8: Create Configuration and Worker Files

8.1 Get the KV Namespace ID

1. Go to Cloudflare Dashboard → Storage & Databases → Workers KV
2. Click on " Your Worker KV "
3. Go to Settings tab
4. Copy Your Namespace ID

8.2 Create wrangler.toml

Bash:
```
notepad wrangler.toml
```

Click **Yes** when asked to create the file. Paste **Click**

Save and Close the Notepad.

8.3 Create the Worker Script

Bash:
```
mkdir src
notepad src\index.js
```

Click **Yes** when asked to create the file. Paste the following worker code **Click**

Save and Close the Notepad.

8.4 Verify Folder Structure

Bash:
```
dir
```

Expected output:

prerender-sitemap-crawler/

├── wrangler.toml

├── package.json

├── package-lock.json

├── node_modules/

└── src/

  └── index.js

```
C:\Users\ADMIN\prerender-sitemap-crawler>dir
 Volume in drive C has no label.
 Volume Serial Number is 90BC-E4DA

 Directory of C:\Users\ADMIN\prerender-sitemap-crawler

29-03-2026  23:37    <DIR>          .
29-03-2026  23:21    <DIR>          ..
29-03-2026  23:24    <DIR>          node_modules
29-03-2026  23:24            34,397 package-lock.json
29-03-2026  23:24               324 package.json
29-03-2026  23:46    <DIR>          src
29-03-2026  23:33               262 wrangler.toml
               3 File(s)         34,983 bytes
               4 Dir(s)  140,826,808,320 bytes free
```

8.5 Deploy Worker

Bash:
wrangler deploy

```
C:\WINDOWS\system32\cmd.
C:\Users\ADMIN\prerender-sitemap-crawler>wrangler deploy

 wrangler 4.78.0

Cloudflare collects anonymous telemetry about your usage of Wrangler. Learn more at https://github.com/cloudflare/worker
s-sdk/tree/main/packages/wrangler/telemetry.md
▲ WARNING The local configuration being used (generated from your local configuration file) differs from the remote co
nfiguration of your Worker set via the Cloudflare Dashboard:

  {
  -  compatibility_date: "2026-03-29"
  +  compatibility_date: "2024-09-02"
  -  preview_urls: false
  +  preview_urls: true
     observability: {
  -    enabled: true
  +    enabled: false
       logs: {
  -      enabled: true
  +      enabled: false
       }
     }
  }


 Deploying the Worker will override the remote configuration with your local one.
```

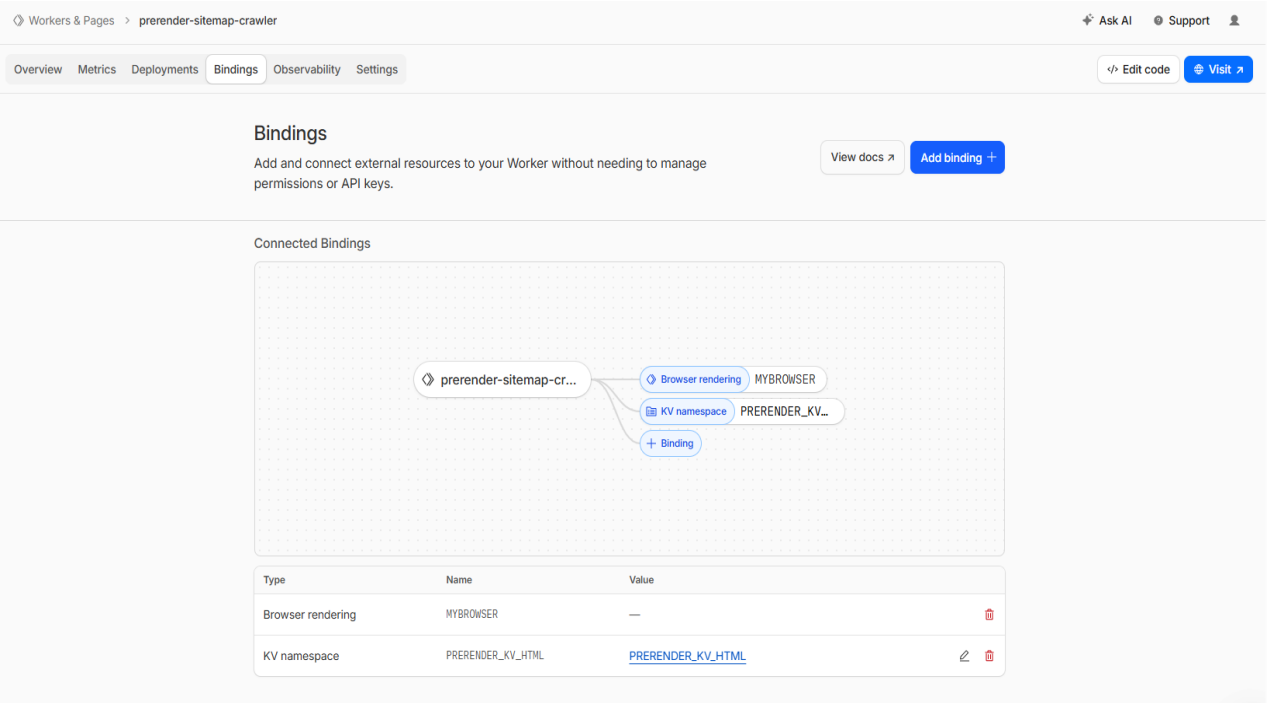
Type **Y** and press Enter.

```
√ Would you like to continue? ... yes
Total Upload: 680.59 KiB / gzip: 133.33 KiB
Worker Startup Time: 13 ms
Your Worker has access to the following bindings:
Binding                                                  Resource
env.PRERENDER_KV_HTML (028c087ee4194135b92276ae64c8acd6)  KV Namespace
env.MYBROWSER                                            Browser

Uploaded prerender-sitemap-crawler (16.88 sec)
▲ [WARNING] Because your 'workers.dev' route is enabled and your 'preview_urls' setting is not in your Wrangler file, Pr
eview URLs will be enabled for this deployment by default.

  To override this setting, you can disable Preview URLs by explicitly setting 'preview_urls =
  false' in your Wrangler file.


Deployed prerender-sitemap-crawler triggers (5.89 sec)
  https://prerender-sitemap-crawler.bwstays.workers.dev
Current Version ID: 55fc5749-02de-4edb-bc82-34cbe772731b
```

Step 9: Add Browser Rendering Binding in Dashboard

Even though wrangler.toml declares the browser binding, it must also be confirmed in the dashboard:

1. Go to **Cloudflare Dashboard → Workers & Pages**
2. Click **prerender-sitemap-crawler**
3. Click **Settings → Bindings → Add**
4. Choose **Browser Rendering**
5. Set Variable name: “Your Variable Name” e.g. MYBROWSER
6. Click **Save**
7. Run wrangler deploy again from terminal



Step 10: Run the Sitemap Crawler in Batches

Why Batches?

Cloudflare Workers have a CPU/execution time limit per request. Running all 300 pages at once caused:

- **Error 1101** — Worker threw exception (resource limit)
- **Error 1102** — Worker exceeded resource limits
- **429** — Browser Rendering rate limit exceeded

Process **5 pages per batch** with a **2-3 minute wait** between each batch to avoid rate limiting.

Expected JSON Response per Batch:

prerender-sitemap-crawler.bwstays.workers.dev/run?batch=59

Pretty-print


```
{
  "totalUrls": 300,
  "totalBatches": 60,
  "currentBatch": 59,
  "processedRange": "URLs 296 to 300",
  "results": [
    {
      "url": "https://www.bwstays.com/assets/img/wildlife/bw-vacation-family-pakshipatalam-wayanad.webp",
      "key": "/assets/img/wildlife/bw-vacation-family-pakshipatalam-wayanad.webp",
      "status": "ok"
    },
    {
      "url": "https://www.bwstays.com/assets/img/wildlife/bw-villas-pakshipathalam-wayanad.webp",
      "key": "/assets/img/wildlife/bw-villas-pakshipathalam-wayanad.webp",
      "status": "ok"
    },
    {
      "url": "https://www.bwstays.com/assets/img/wildlife/bw-weekday-pakshipathalam-bird-sanctuary-wayanad.webp",
      "key": "/assets/img/wildlife/bw-weekday-pakshipathalam-bird-sanctuary-wayanad.webp",
      "status": "ok"
    },
    {
      "url": "https://www.bwstays.com/assets/img/wildlife/bw-work-vacation-pakshipatalam-wayanad.webp",
      "key": "/assets/img/wildlife/bw-work-vacation-pakshipatalam-wayanad.webp",
      "status": "ok"
    },
    {
      "url": "https://www.bwstays.com/assets/img/icons/bw.svg",
      "key": "/assets/img/icons/bw.svg",
      "status": "ok"
    }
  ],
  "nextStep": "✅ All pages completed!"
}
```

How to Run Each Batch!

Visit each URL in your browser, waiting for the full JSON response before proceeding:

For e.g.:

Batch 0: https://prerender-sitemap-crawler.bwstays.workers.dev/run?batch=0

Batch 1: https://prerender-sitemap-crawler.bwstays.workers.dev/run?batch=1

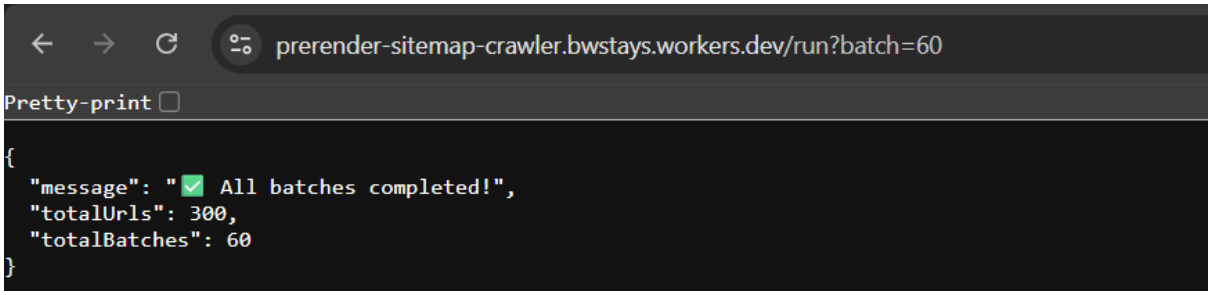
Batch 2: https://prerender-sitemap-crawler.bwstays.workers.dev/run?batch=2

…….

Batch 59: https://prerender-sitemap-crawler.bwstays.workers.dev/run?batch=59

Run in this way in each tab.

Finally, the expected response will be:



Safe Batch Workflow

1. Open the batch URL in browser
2. Wait for full JSON response to appear
3. Confirm results show "status": "ok"
4. Wait **2–3 minutes** before opening next batch
5. Close the tab safely
6. Open next batch URL

Step 11: Verify KV is fully Populated

1. Go to **Cloudflare Dashboard → Storage & Databases → Workers KV**
2. Click **PRERENDER_KV_HTML**
3. Click **KV Pairs** tab
4. Confirm All you entries are present

Performance Benchmarks

The following benchmarks reflect real-world observations using Cloudflare Browser Rendering on the Workers Free plan. Times vary based on page complexity and JavaScript bundle size.

Crawl Time by Site Size

Pages	Batches (5/batch)	Avg per Page	Total Crawl Time	KV Size (est.)
100	20	8–12s	~45–60 min	~50 MB
200	40	8–12s	~90–120 min	~100 MB
500	100	8–12s	~4–5 hours	~250 MB

Note: Total crawl time includes the mandatory 2–3 minute wait between batches. On a paid plan with higher concurrency limits, these times can be reduced significantly.

Static vs Dynamic Site Comparison

Factor	Static HTML	SPA (no prerender)	SPA + This System
Googlebot crawl success	100%	20–60%	95–100%
Time to first byte (bot)	< 100ms	< 100ms (empty!)	< 5ms (cached)
Content visible to bot	Full	None	Full
Social media previews	✓ Work	× Broken	✓ Work
Infrastructure cost	Minimal	Minimal	Free tier viable

Why 5 Pages Per Batch Works Well on Free Plan

- Each page takes 8–15 seconds to render — 5 pages fits within the 30-second Worker execution limit
- One browser per batch avoids concurrency limit errors (free plan: 2 concurrent max)
- 2–3-minute cooldown lets the rate limit window reset between batches
- Batch size can be increased to 10 on a paid plan with a shorter 60-second wait

Common Pitfalls & Solutions

Error	Cause	Solution
Error 1101 — Worker threw exception	Single page crash mid-batch	Wrap every page in individual try/catch; always close page in finally block
Error 1102 — Resource limits	Too many pages per batch	Reduce BATCH_SIZE to 3–5 pages
429 — Rate limit exceeded	Too many browser launches	Wait 3–5 minutes between batches; reduce concurrency
Module not found: @cloudflare/puppeteer	Deployed from dashboard editor	Must deploy via Wrangler CLI with npm package installed locally
Empty HTML returned	Page requires auth or JS fails	Check page loads in browser; skip auth-gated pages
KV key not matching URL	Path normalization issue	Normalize: lowercase, strip trailing slash, map / to /index
Prerender loop	Worker calling itself	Always check for X-Prerender header before rendering
'There is nothing here yet'	Wrong worker deployed	Check wrangler.toml name matches intended worker exactly
Browser time limit exceeded	Daily 10-min free quota hit	Wait until next UTC midnight; use usage tracking to stop before limit

Competitor Landscape & Cost Analysis

Before we close, it is worth understanding exactly how much money you are saving by building this system yourself. The prerendering market has several established players each with meaningful monthly costs. Here is a complete breakdown of every major competitor, their pricing, and how they compare to the Cloudflare-native approach you have just built.

The Prerendering Market in 2026 The JavaScript prerendering market has consolidated around a handful of managed services. Most charge based on the number of renders (cached page snapshots) per month and how frequently those caches are refreshed. Prices have risen sharply Prerender.io removed its free tier in October 2025 and implemented a pricing restructure that some customers reported as a 250%+ increase.

Key market shift:

As of October 15, 2025, Prerender.io discontinued its unlimited free plan (1,000 renders/month) and replaced it with a 30-day trial only. New users must subscribe to a paid plan after the trial ends. This makes self-hosted solutions significantly more attractive for cost-conscious teams

Competitor Pricing Comparison (2026)

Service	Type	Entry Price	100 pages/mo	300 pages/mo	1,000 pages/mo
Prerender.io	Managed SaaS	$49/mo (Starter)	$49/mo	$49/mo	$99–$349/mo
SEO4Ajax	Managed SaaS	Free (1K pages)	Free	$39/mo	$132/mo
SnapSearch	Usage-based SaaS	Free (1K snaps)	~$0.20/mo	~$0.60/mo	~$2.00/mo
LovableHTML	Managed SaaS	$16/mo (10K)	$16/mo	$16/mo	$41/mo
Netlify Prerender	Platform add-on	$9/mo (1K credits)	$9/mo	$9/mo	$9–$27/mo
Rendertron	Self-hosted OSS	Server cost only	$5–20/mo	$10–40/mo	$20–80/mo
AWS Lambda + Puppeteer	Self-hosted	Pay-per-use	$1–5/mo	$3–15/mo	$10–50/mo
This System (CF Workers)	Self-hosted edge	$0 (free tier)	$0	$0	$0–5/mo

5-Year Total Cost of Ownership Analysis

To truly appreciate the savings, consider what you would spend over 5 years using a managed service versus this self-built Cloudflare system. The following assumes a medium-sized website with 300 pages that refreshes cache monthly.

Solution	Monthly Cost	Year 1	Year 3	Year 5	5-Year Total
Prerender.io (Starter)	$49	$588	$1,764	$2,940	$2,940
Prerender.io (Growth)	$99	$1,188	$3,564	$5,940	$5,940
SEO4Ajax (Project)	$39	$468	$1,404	$2,340	$2,340
LovableHTML (Starter)	$16	$192	$576	$960	$960
AWS Lambda + Puppeteer	$10	$120+	$360+	$600+	$600+
Cloudflare Workers (This System)	$0	$0	$0	$0	$0

5-Year Saving vs Prerender.io Starter: $2,940 | vs **Prerender.io Growth:** $5,940 | vs **SEO4Ajax:** $2,340

The Engineering Time Break-Even

There is one honest cost to acknowledge with this self-built approach: your engineering time. Here is how to think about whether the DIY route makes financial sense for your team:

Task	One-time Setup Time	Ongoing Maintenance
Follow this guide end-to-end	3–5 hours	0 hours (fully automated)
Run initial sitemap crawl	2–4 hours	1 hour/month (re-crawl new pages)
Monitor KV usage in dashboard	15 minutes	5 min/month
Total engineering investment	5–9 hours	~1 hour/month

If you have a developer on your team who can follow this guide, the Cloudflare Workers approach pays for itself within 60 days and saves $500–$6,000 over 5 years depending on which paid service you would have used. The only scenario where a paid service is clearly worth it is when you have zero developer access, strict data residency requirements, or need AI search visibility monitoring built in.

The key advantages of this approach:

- Zero external dependencies →no Prerender.io, no Render Tron, no third-party services
- Edge-native delivery →HTML served from Cloudflare's global network in under 5ms
- Cost-effective → entirely free for most small-to-medium sites on the Workers free plan
- Resilient →on-demand headless rendering catches any pages missed by the initial crawl
- Maintainable → two focused workers with clear, single responsibilities

Conclusion

Building a prerendering system using Cloudflare Workers, Puppeteer, and KV Storage provides a powerful, scalable, and cost-efficient solution to solve SEO challenges in modern JavaScript-heavy applications. By leveraging edge computing, we can serve fully rendered HTML to bots instantly, improving search visibility and performance without modifying the core frontend architecture.

One of the biggest advantages of this approach is its ability to run within the **free tier limits**. With **100,000 Worker requests per day**, **10 minutes of browser rendering time**, and **1 GB of KV storage**, it is more than sufficient to prerender and serve **hundreds of pages at zero cost**. This makes it an ideal solution for startups, personal projects, and even medium-scale applications looking for a budget-friendly SEO optimization strategy.

Overall, this system demonstrates how modern serverless tools can be combined to build a **production-ready, scalable prerendering pipeline** without relying on expensive third-party services.